



Product Engineering Build Challenge

A live build interview for payments product engineers



Why a Live Build Interview?

- Leetcode-style and take-home challenges fail to assess 2026 development realities.
- The product engineering role requires understanding business problems, shipping rapidly, while retaining deep technical comprehension.

The Build Challenge



You are a product engineer at VGS. Our design-partner customer is **Atlas Voyages**, a travel aggregation platform that sells airline tickets, hotel stays, and tours & experiences from suppliers worldwide all in one checkout.

Their payment traffic is currently regional to the US and EU, but they are expanding to east Asia as well.

1. **Atlas processes cards through different payment vendors in different regions** each with its own API, its own formats, and its own failure modes. Every new vendor is a months-long manual integration project for Atlas's engineers who were hired to build travel software, not payments plumbing.
2. **Travel inventory is perishable.** A fare hold or room rate lives for minutes; when a charge fails at one vendor, the booking usually dies with it. Atlas wants failed charges automatically retried through another vendor before the fare expires.
3. **Refunds and reporting** – due to cancelled flights, changed itineraries, weather. Atlas's finance team can't answer "how much did we take today, net of refunds, across all vendors and currencies?" without stitching together several vendor dashboards and CSV exports.

Atlas has asked VGS to solve this once, for good: **one connection**. Atlas integrates a single standard API for charges and refunds. Behind it, VGS routes each payment to the right vendor, fails over when a vendor is down, and keeps an accurate ledger of every movement of money. The goal is that Atlas's engineers never hand-configure a vendor integration again.

Atlas was able to provide API specs for each of these vendors, but they don't have actual accounts that can be shared. Nevertheless, you are responsible to build the MVP of that product.



Core Assessment Pillars

- **AI Collaboration:** Specifying before prompting; catching AI errors.
- **Technical Depth:** Explaining architecture, tracing requests end-to-end; defending every line of code.
- **Business Sense:** Identifying common payments traps (idempotency, partial failures, integer currency).
- **Requirements Discovery:** Asking clarifying questions about business use-cases.